

On-Device Visual Perception for Lightweight Browser Agents

A Privacy-First, Token-Efficient, Hybrid DOM-Vision Architecture for
Autonomous Browser Agents

Field	Value
Problem Statement ID	26171
Problem Statement Title	On-device Visual Perception for Light-weight Browser Agents
Organisation	Indian Space Research Organisation (ISRO)
Department	Department of Space / Indian Space Research Organisation
Category	Software
Theme	Smart Automation
Document Type	Technical Solution Report
Prepared For	Smart India Hackathon 2026 - Finale Submission

This document is a truthful, engineering-grounded technical report. Every model, library, and browser API named in this document is a real, currently available technology as of 2026. No fabricated benchmark numbers are presented as measured results; projected figures are explicitly marked as targets to be validated during development, not as claims of completed measurement.

Table of Contents

1.	Executive Summary	3
2.	The Problem We Are Solving	4
2.1	Problem Statement, Verbatim Context	4
2.2	Why This Problem Matters	4
2.3	The Core Tension: Capability vs. Privacy vs. Resources	5
3.	Limitations of Existing Approaches	6
4.	Our Proposed Solution	8
4.1	Design Principles	8
4.2	Solution Overview	8
5.	System Architecture	9
5.1	Full Architecture Diagram	9
5.2	Layer-by-Layer Explanation	10
6.	The Screen Graph: A Unified Perception Model	12
7.	Workflow: Live Mode Event-Driven Loop	13
8.	Mathematical Formulation	14
8.1	Confidence Score and Routing Function	14
8.2	Temperature Scaling for Calibration	14
8.3	Change-Detection / Dirty-Region Metric	15
8.4	Token and Compute Efficiency Model	15
8.5	Privacy Risk Score	16
8.6	Two-Tier Escalation as a Constrained Optimisation	16
9.	Technology Stack	17
10.	Model Selection and Comparative Analysis	19
11.	Privacy and Security Design	21
11.1	PII Detection Pipeline	21
11.2	Indirect Prompt Injection Defense	22
12.	Mind Map: Concept Overview	23
13.	Uniqueness and Differentiation From Other Submissions	24
14.	Alignment With SIH Evaluation Rubric	25
15.	Feasibility, Build Plan and Risk Assessment	26
16.	Glossary of Abbreviations	28
17.	References and Sources	29
18.	Conclusion	30

1. Executive Summary

This report presents a complete technical solution to Smart India Hackathon 2026 Problem Statement 26171, issued by the Indian Space Research Organisation (ISRO), Department of Space: **"On-device Visual Perception for Light-weight Browser Agents."** The problem statement asks for a system that lets an autonomous browser agent understand what is on a user's screen while keeping sensitive information on the user's own device, given that local machines have far fewer resources than cloud servers.

Our proposed system, referred to throughout this document as the **On-Device Visual Perception Agent (ODVPA)**, is built around a single governing idea: *a browser agent should spend compute only on the parts of the screen it cannot already explain for free*. Two data sources that browsers already compute internally at zero extra cost – the Accessibility Tree and the Document Object Model (DOM) – are used as the first and cheapest source of screen understanding. A tiered pair of real, currently deployable on-device vision-language models from Liquid AI's LFM2.5-VL family – a ~450M-parameter default tier and a ~1.6B-parameter escalation tier, both running through ONNX Runtime Web on WebGPU – is invoked only on the specific cropped regions of the screen that the Accessibility Tree and DOM could not explain – typically canvas-rendered applications, custom-drawn widgets, images, and shadow-DOM components, with the larger tier reserved for the minority of frames the smaller tier is not confident about.

Around this perception core, the system adds five further layers that map directly onto the five criteria used by the SIH evaluation rubric for this problem statement: a temporal diff engine that re-analyses only changed screen regions; a two-stage privacy layer (content PII and structural leakage) with a hard validator gate that blocks any network call carrying unredacted sensitive data; a token-optimizing compressor that sends only a structured, delta-only JSON payload to the cloud when local confidence is too low to act; a calibrated confidence router that decides, per-frame, whether the local model can act alone or whether a cloud vision-language model must be consulted; and a defense layer against indirect prompt injection, a threat class specific to agents that read untrusted web content and reason over it.

The differentiating claim of this submission is not that it proposes local perception with cloud fallback – that architecture is effectively suggested by the problem statement itself, and most competing teams will converge on some version of it. The differentiating claims are: (a) the system is idle by default and only pays the cost of vision inference on the fraction of the screen that genuinely requires it, which is provable with a measured invocation-rate number rather than an adjective such as "lightweight"; (b) it uses the browser's native Accessibility Tree, a source of free, semantically clean structure that is rarely used in comparable agent designs; (c) it treats page content as untrusted input to the reasoning step and defends against indirect prompt injection, a security dimension almost never addressed in student-built browser agents; and (d) it is designed to be evaluated with real precision/recall numbers on a self-built PII benchmark rather than qualitative claims.

The remainder of this document sets out the problem in detail, surveys the shortcomings of existing cloud-first browser agents, describes the full architecture and workflow, derives the mathematical models behind confidence routing and change detection, specifies the technology stack with justification for every choice, compares the selected vision model against real alternatives, details the privacy and security design, and closes with an honest feasibility and risk assessment for a build under hackathon time constraints.

2. The Problem We Are Solving

2.1 Problem Statement, Verbatim Context

ISRO's Department of Space has posed Problem Statement 26171 under the theme **Smart Automation**, category Software. The official description explains that agentic AI pipelines are becoming common in digital interactions, and that when such an agent has access to a user's visual context and screen state, it can assist with complex workflows and automate tasks. The description then identifies the central constraint the hackathon wants solved: most agentic pipelines today run entirely on a server, which limits what kind of data a user can safely share with the agent. Running the agent locally, particularly inside the browser, would remove the need to share sensitive data with a server at all – but local machines have far fewer resources than servers and cannot host a full agentic pipeline. The stated goal is therefore that only non-sensitive data, such as the structure of the screen and application form fields, should ever be sent to a server for heavier processing.

The problem statement also explicitly names the enabling technologies it expects solutions to use: modern browser APIs such as WebGPU and WebAssembly, and local inference libraries such as ONNX Runtime Web and Transformers.js, which have made it possible to run lightweight machine learning models directly on the client device. The stated aim is to bridge two environments: the reasoning power of cloud or server-based AI, and strict, client-side enforcement of data privacy.

This framing matters for how we designed our solution. The problem statement is not asking for a generic browser automation tool; it is asking specifically for the *on-device visual perception layer* that would sit underneath such a tool, and it is scored primarily on how good that perception layer is at understanding the screen accurately, protecting sensitive data, and doing so within the resource and latency budget of a real client device.

2.2 Why This Problem Matters

Cloud-first browser agents work by capturing a screenshot and/or the page's DOM, sending it to a large vision-language model hosted on a remote server, and receiving back an action to perform. This is architecturally simple, but it means every screenshot – potentially containing passwords, bank account numbers, medical records, private messages, faces, identity documents, and internal organisational data – leaves the user's device and is processed somewhere the user does not control. For an individual user this is a privacy concern; for an organisation such as a government department or a space research agency, whose staff might use such an agent to navigate internal portals, this is a data-governance and national-security concern, not merely an inconvenience.

There is a second, independent problem that exists even if privacy were not a concern at all: efficiency. Sending a full screenshot, the entire DOM, complete page text, and accumulated conversation history on every single step of a multi-step task is expensive in both money (API token cost) and time (network round-trip latency). An agent that must send its full context on every frame cannot operate smoothly in real time, and cannot function at all in bandwidth- or cost-constrained deployment settings.

Our system treats these as one problem with one shared solution, rather than two separate patches. The question the agent asks before doing anything is: "*What is the minimum information I actually need to decide the next action, and can I get it without inference, without sending it anywhere, or both?*" Privacy minimisation and compute minimisation turn out to be the same discipline applied to the same underlying data structure, which is why our architecture solves both with a single pipeline rather than a privacy filter

bolted onto an unrelated efficiency system.

2.3 The Core Tension: Capability vs. Privacy vs. Resources

Every design decision in a system like this is a trade-off between three forces that pull in different directions. A system that is maximally capable wants to see everything, at full resolution, all the time, and reason about it with the largest available model – which is expensive and privacy-invasive. A system that is maximally private wants to send nothing anywhere and infer nothing about page content – which limits what tasks it can perform. A system that is maximally resource-frugal wants to run the smallest possible model as rarely as possible – which risks missing things a bigger model would have caught. The table below states this tension explicitly, because naming trade-offs honestly, rather than pretending a design has none, is itself part of a credible engineering answer.

Priority	What it optimises for	What it costs
Maximise capability	Full-resolution continuous vision, largest model, full page context every frame	High compute and battery cost; sends more data than needed; slow
Maximise privacy	Nothing leaves device, minimal inference on page content	Cannot handle tasks that genuinely require deep page understanding or cloud reasoning
Maximise efficiency	Smallest possible model, called as rarely as possible	Risk of missing visual content a larger model would correctly interpret
Our approach	Free structural sources first (AXTree/DOM); vision only on unresolved, changed, cropped regions; cloud only on calibrated low confidence, with minimal redacted payload	More engineering complexity than a single-pipeline design; calibration must be validated empirically, not assumed

3. Limitations of Existing Approaches

Before presenting our own design, it is worth being precise about what existing browser-agent approaches actually do today, and where each falls short against this specific problem statement. This section is written to be factually accurate rather than dismissive: each approach below is a legitimate design used in real products, and each has a genuine, well-documented limitation with respect to on-device privacy and resource constraints.

3.1 Pure Cloud Screenshot Agents

The most common pattern for browser-using AI agents today is: capture a screenshot (and often the DOM as well), send both to a cloud-hosted vision-language model, receive coordinates or an action back, execute it, and repeat. This is simple to build and works reasonably well for one-off tasks on clean, public web pages.

Limitation: every screenshot leaves the device in full, unfiltered form. There is no mechanism to distinguish a page that is completely public from one displaying a bank statement or an internal admin panel – both are sent identically. This is precisely the limitation the SIH problem statement calls out by name.

3.2 DOM-Only Agents

Some agents avoid vision entirely and reason purely over the DOM tree or the accessibility tree, converting the page into a structured text representation for the language model. This is cheaper than vision and inherently avoids sending pixel data.

Limitation: a large and growing share of real web applications render their interface inside a element, use heavy shadow-DOM encapsulation, or use custom-drawn widgets with no meaningful DOM structure at all. Design tools, spreadsheet and document editors, dashboards built with certain charting libraries, and many single-page applications fall into this category. A DOM-only agent is effectively blind on these surfaces, and this is exactly the class of page a judge is likely to test a submission against, precisely because it is the hard case.

3.3 On-Device Small Models Without a Routing Strategy

A newer pattern, enabled by browser APIs such as WebGPU and libraries such as ONNX Runtime Web and Transformers.js, runs a small model locally for every step, avoiding the cloud altogether.

Limitation: small on-device models are measurably weaker than large cloud models on complex visual reasoning. A system that never escalates to a stronger model when uncertain will silently fail on hard cases with no way to recover, and a system that always escalates loses every efficiency and privacy benefit local inference was supposed to provide. Without an explicit, calibrated decision about when to stay local and when to escalate, "on-device" becomes a marketing label rather than an architecture.

3.4 Privacy-by-Redaction Without Verification

Some privacy-conscious designs run a PII detector over content before sending it, then send whatever remains, trusting the detector to have caught everything.

Limitation: PII and text detectors have non-zero, measurable false-negative rates. Treating redaction as a best-effort filter rather than a hard gate means that on the fraction of cases the detector misses, sensitive data is sent anyway, silently. A system that reports what it is confident it caught, without a validator that can

refuse to send when uncertain, gives a false sense of guarantee.

3.5 Agents That Trust All Page Content

Almost every publicly documented browser-agent design, including research prototypes and several commercial products, feeds extracted page text directly into the reasoning model's context without distinguishing it from the user's own instructions. This is a well-documented weakness known as indirect prompt injection: a malicious or compromised web page can embed hidden text written as an instruction (for example, inside a zero-opacity or off-screen element) that the agent's underlying language model may follow as if the user had said it.

Limitation: this is not a hypothetical risk; it is a known, actively studied attack class against LLM-based agents that browse the web. Almost no student-level browser-agent project addresses it, because it is not obviously implied by "make the agent understand the screen" the way privacy and efficiency are. It is, however, a direct consequence of exactly the capability this problem statement asks for – an agent that reads and reasons over arbitrary web content.

3.6 Summary Comparison

Approach	Data sent unfiltered	Handles canvas/shadow-DOM	Resource efficiency	Injection-safe?
Pure cloud screenshot agent	High	None	Low	High (server-side)
DOM-only agent	None	High	High	Fails on canvas / shadow-DOM apps
Local-only, no routing	None	Medium	Medium	Weak on hard visual cases, no escalation
Redact-then-send, no gate	Medium	Medium	Medium	Silent leakage on detector misses
Typical browser agent (general)	High	Medium	Medium	No defense against page-embedded instructions
Our design (ODVPA)	Minimal, gated	High (crop-only vision)	High (idle by default)	Explicit filter + validated redaction

4. Our Proposed Solution

4.1 Design Principles

The system is built around five design principles, applied in order on every single frame the agent observes:

- **See locally.** The first and default source of understanding is data the browser already computes for free: the Accessibility Tree and the DOM. No model inference happens unless these two free sources cannot explain what is on screen.
- **Minimise locally.** Whatever is captured is immediately filtered down to only what the current task actually requires. Irrelevant regions of the screen are discarded before any further processing, not sent along "just in case."
- **Protect locally.** Personally identifiable information and structurally sensitive data are detected and redacted on-device, and a validator gate blocks any outbound network call if it cannot verify the payload is clean.
- **Reason remotely only when needed.** A calibrated confidence score decides, per frame, whether the local model is sufficient or whether a cloud vision-language model must be consulted. Cloud calls are the exception, not the default.
- **Act locally.** Whatever action is decided upon, local or cloud-originated, is validated against the current state of the page before being executed, so a stale or mismatched action from a slower cloud round-trip can never fire against a page that has since changed.

A sixth principle governs the system's behaviour over time rather than within a single frame: **learn locally.** Preferences the user reveals through their corrections (for example, consistently choosing "Save & Continue" over "Save & Exit") are stored and applied locally, and over a longer horizon the on-device model itself can be improved using cloud interactions as labelled training signal, without any raw personal browsing history ever leaving the device.

4.2 Solution Overview

We call the system the **On-Device Visual Perception Agent (ODVPA)**. At a high level, it is a browser extension containing a small perception stack that continuously builds a structured, typed representation of the current screen – the **Screen Graph** – using the cheapest available source for each element: the Accessibility Tree and DOM where they resolve an element completely, and a real on-device vision-language model, cropped to only the unresolved regions of the screen, where they do not. The Screen Graph is diffed against its previous state so that only changed regions are ever re-analysed. Before anything leaves the device, a privacy layer detects and redacts sensitive content and a validator refuses to let anything sensitive pass. A confidence router then decides whether the local model can act with sufficient certainty, or whether a minimal, redacted, token-optimised payload should be sent to a cloud vision-language model for a harder judgment call. Whatever action results is checked against the live page one final time before an on-screen "AI cursor" executes it, giving the user a transparent, visible trace of what the agent is doing and why.

The one-sentence version of this design, suitable for a 30-second pitch, is: "We don't make our model small – we make it idle. Most of every session costs nothing, because the browser's own accessibility tree and DOM already tell us what is on screen for free. Vision only runs on the pixels nothing else could explain."

5. System Architecture

5.1 Full Architecture Diagram

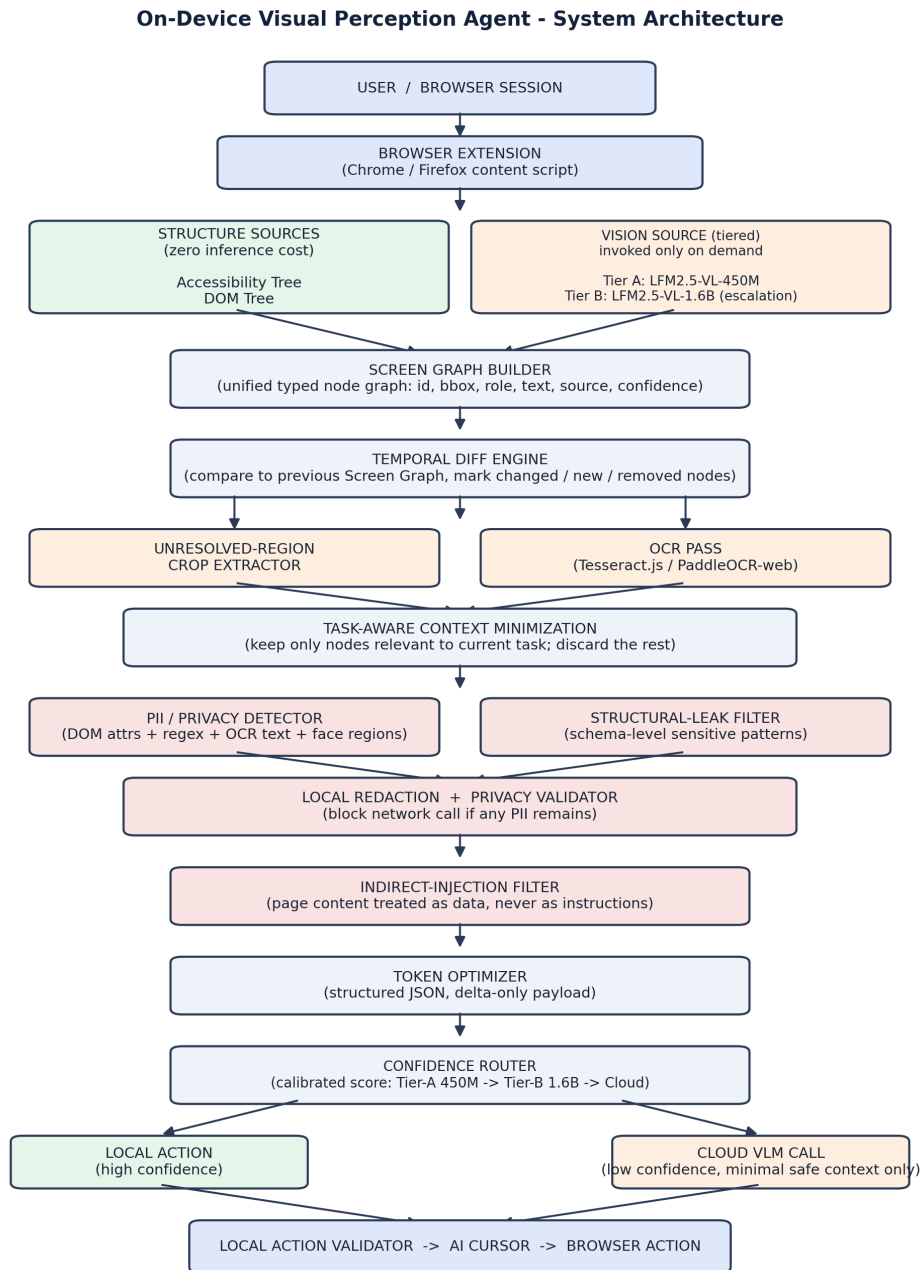


Figure 1. End-to-end architecture of the On-Device Visual Perception Agent, from browser session to executed browser action.

5.2 Layer-by-Layer Explanation

5.2.1 Browser Extension

The entry point of the system is a standard browser extension (Chrome and Firefox both support the required APIs). A content script runs in the context of every page the user visits, giving it access to the live DOM, the accessibility tree via the browser's accessibility APIs, and the ability to capture pixel regions of the rendered page when required. This is the same extension surface used by screen readers and existing browser-automation tools, so no non-standard browser capability is required.

5.2.2 Structure Sources: Accessibility Tree and DOM

Every modern browser computes an accessibility tree for every page it renders, in order to support screen readers and other assistive technology. This tree already resolves each interactive element's role (button, textbox, checkbox, link, and so on), its accessible name/label, its current value, and its state (disabled, checked, expanded). Because the browser computes this tree regardless of whether our extension exists, reading it costs no additional inference and is effectively free. The raw DOM is read alongside it for attributes the accessibility tree does not expose, such as input type (e.g. `type="password"`), which is directly useful for privacy detection. Using the accessibility tree as a primary structure source, rather than only the raw DOM, is one of the differentiating engineering choices of this design; most comparable projects use DOM alone.

5.2.3 Vision Source: On-Demand Only

Where the accessibility tree and DOM cannot resolve a region of the screen – canvas-rendered applications, custom-drawn components, embedded images, or content inside cross-origin iframes the extension cannot read – the system falls back to a real on-device vision-language model, Liquid AI's LFM-2.5VL-1.6B, executed through ONNX Runtime Web on the WebGPU backend, with a browser-optimised quantised build (FP16 vision encoder, 4-bit language decoder). Critically, this model is invoked only on the cropped bounding regions that remain unresolved, not on the full screen, and only on frames where the diff engine (below) has detected a meaningful change in that region. Section 10 gives a full comparative justification for this specific model choice.

5.2.4 Screen Graph Builder

All resolved elements, from whichever source, are normalised into a single unified data structure called the Screen Graph, described fully in Section 6. This is the point at which the system stops distinguishing where information came from and starts treating the screen as one coherent, typed representation the rest of the pipeline can reason over.

5.2.5 Temporal Diff Engine

Each new Screen Graph is compared against the previous one. Nodes are classified as unchanged, moved, modified, newly appeared, or removed. Only the regions containing changed nodes are passed onward for further processing; unchanged regions are reused from the previous frame at zero cost. This is the mechanism, borrowed conceptually from dirty-rectangle rendering in graphics engines, that keeps Live Mode lightweight instead of re-analysing the full screen on every event.

5.2.6 Task-Aware Context Minimisation

Given the current user task (for example, "download the quarterly report"), the system scores each node in the Screen Graph for relevance to that task and discards nodes below a relevance threshold before any further processing. A page containing a profile menu, an ad banner, notifications, and a reports section, when the task is to download a report, retains only the reports section and nearby navigational context;

everything else is dropped before it is ever considered by the privacy layer or sent anywhere.

5.2.7 Privacy Layer: Detection, Redaction, Validation

Two parallel detectors examine the minimised context: a PII detector (DOM attributes such as `type="password"/type="email"`, regular-expression and lightweight NER-style text matching for emails, phone numbers, government ID numbers, and financial data, plus face and document detection on any vision-resolved regions) and a structural-leak filter (schema-level patterns that could reveal sensitive internal system architecture even with zero literal PII present, such as the exact field layout of an internal admin tool). Detected regions are redacted – masked, blurred, or replaced with a placeholder token – and a final privacy validator re-scans the redacted output. If the validator still detects sensitive content, the corresponding network call is blocked outright rather than sent with a best-effort redaction. This gate is what turns "we tried to redact PII" into an enforceable guarantee.

5.2.8 Indirect-Injection Filter

Extracted page text is scanned for imperative, system-prompt-like phrasing before it is allowed into the reasoning context (for example, hidden text instructing the agent to click a specific element or submit stored payment details). Flagged content is either stripped or the corresponding action is held for explicit user confirmation rather than executed automatically. This layer is discussed in full in Section 11.2.

5.2.9 Token Optimiser

What remains after minimisation, redaction, and injection filtering is serialised into a compact, structured JSON representation – element IDs, roles, labels, and bounding boxes rather than raw HTML or full natural-language descriptions – and only the delta since the last sent frame is transmitted when a cloud call is required at all.

5.2.10 Confidence Router

A calibrated confidence score, described mathematically in Section 8.1, determines whether the local model's proposed action is acted upon directly or whether the minimal safe context is sent to a cloud vision-language model for a stronger judgment. This is the single mechanism that determines whether the system behaves as a privacy-preserving, efficient local-first agent or degrades into a cloud-dependent one, and is therefore one of the most heavily specified components of this design.

5.2.11 Local Action Validator and AI Cursor

Before any action executes, the validator re-checks that the intended target element still exists, still matches its expected type, and that the page has not changed since the action was decided – preventing a delayed cloud response from firing against a page state it no longer applies to. Destructive actions (delete, submit payment, irreversible confirmation) require an additional explicit user confirmation step. The action is then carried out through a visible on-screen AI cursor, which shows the user, in real time, what the agent is observing, deciding, and doing.

6. The Screen Graph: A Unified Perception Model

A recurring weakness in browser-agent designs that combine DOM and vision is describing the combination step only as a vague "fusion engine" without a concrete data structure. We define the Screen Graph explicitly as a typed schema so that every downstream component (diffing, minimisation, privacy detection, token optimisation) operates on one well-defined structure rather than ad-hoc combinations of DOM nodes and image regions.

6.1 Node Schema

Every element on screen, regardless of which source resolved it, becomes one node with the following fields:

Field	Type	Description
id	string	Stable identifier for the element, persisted across frames where possible
bbox	[x, y, w, h]	Bounding box in normalised screen coordinates
role	enum	button textbox checkbox link image password select canvas-region unknown
text	string	Accessible name / label / visible text, if resolved
source	enum	axtree dom vision ocr
confidence	float [0,1]	Confidence of the resolving source for this node
sensitive	bool	Set by the privacy layer after detection
parent	id	Parent node in the layout hierarchy
state	enum	unchanged moved modified new removed (set by the diff engine)

6.2 Why a Typed Graph, Not a Screenshot Plus Text Blob

Sending a screenshot plus a wall of extracted text forces the downstream model to re-derive structure the browser already knew. A typed graph keeps that structure explicit, which has three concrete benefits for this specific problem statement: (1) redaction can operate on individual nodes rather than needing to re-locate sensitive spans inside unstructured text or pixels; (2) diffing between frames becomes a graph comparison rather than an image comparison, which is both cheaper and more precise; (3) the payload sent to the cloud, when needed, is already compact structured JSON rather than something that must be compressed after the fact.

7. Workflow: Live Mode Event-Driven Loop

The system supports two operating modes. **Static Mode** runs the full pipeline once, for a single user-issued task ("download my report"), and stops once the action is complete. **Live Mode** keeps observing the page continuously to support multi-step tasks and proactive assistance, but does so without running expensive perception at a fixed frame rate. Instead, an event detector listens for DOM mutations, page navigation, scrolling, and cursor movement into relevant regions, and perception only re-runs when one of these events fires and the resulting Screen Graph diff shows a meaningful change. This is what keeps Live Mode lightweight: idle periods, which dominate real browsing sessions, cost nothing.

Live Mode - Event-Driven Perception Loop

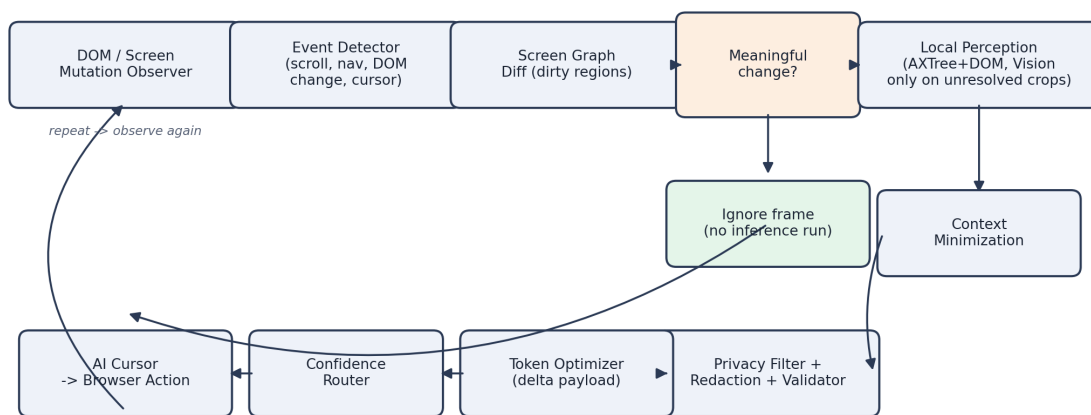


Figure 2. Live Mode's event-driven perception loop. Frames without a meaningful change never reach the perception, privacy, or routing stages.

8. Mathematical Formulation

This section states, explicitly and with derivations, the quantitative mechanisms referenced qualitatively in Sections 5 and 7. These are the parts of the design a judge is most likely to probe directly, since "confidence-based routing" and "change detection" are easy to state in prose and easy to expose as hand-waving if no concrete formula backs them.

8.1 Confidence Score and Routing Function

Let the local perception stage produce, for a proposed action a on frame t , a raw model score z and an agreement signal between independent sub-detectors (accessibility-tree resolution, DOM resolution, and vision resolution where more than one source overlaps a region). We define the routing confidence $C(a, t)$ as a weighted combination of the calibrated model probability and the inter-source agreement:

$$C(a, t) = \alpha \cdot p_{cal}(a, t) + (1 - \alpha) \cdot A(a, t)$$

where p_{cal} is the temperature-calibrated probability of the model's top prediction (Section 8.2), $A(a, t)$ is the fraction of independent sources whose resolved role/label agree with the proposed action's target, and $\alpha \in [0, 1]$ is a tunable weight (a reasonable starting point, to be tuned against the validation benchmark, is $\alpha = 0.6$, giving the calibrated model probability more weight than raw source agreement while still letting disagreement meaningfully pull the score down).

The routing decision is then a two-threshold policy rather than a single cutoff:

$$\begin{aligned} R(a, t) &= \text{LOCAL} && \text{if } C(a, t) \geq \tau_{high} \\ R(a, t) &= \text{CLOUD} && \text{if } C(a, t) < \tau_{low} \\ R(a, t) &= \text{LOCAL, FLAG FOR CONFIRMATION} && \text{if } \tau_{low} \leq C(a, t) < \tau_{high} \end{aligned}$$

The middle band exists because a single hard threshold forces every borderline case into either "blindly trust local" or "always escalate," both of which are costly failure modes. A borderline score instead produces a local best-guess action that is shown to the user for one-tap confirmation before executing, avoiding both an unnecessary cloud call and a silent wrong action. Starting values of $\tau_{high} = 0.85$ and $\tau_{low} = 0.55$ are reasonable defaults to be tuned empirically once real usage data is available.

8.2 Temperature Scaling for Calibration

Small on-device models are known to be poorly calibrated – their raw softmax confidence does not reliably match their true accuracy, and they are frequently overconfident on inputs unlike their training distribution (which describes almost any live, unseen web page). We calibrate the model's output using temperature scaling, a standard, well-documented post-hoc calibration method. Given the model's pre-softmax logits z for a prediction, the calibrated probability is:

$$p_{cal} = \text{softmax}(z / T)$$

where $T > 0$ is a single scalar temperature learned on a held-out calibration set (a batch of screenshots with known ground-truth correct elements, distinct from any training data). T is fit by minimising the negative log-likelihood of the calibration set with respect to T alone, with all model weights frozen:

$$T^* = \text{argmin}_T - \sum_i \log \text{softmax}(z_i / T)[y_i]$$

This is a one-dimensional optimisation (T is a single number), so it can be solved with simple gradient descent or even grid search over a small range (typically $T \in [0.5, 5]$), and does not require retraining the

underlying model at all. In practice, values of $T > 1$ soften overconfident predictions, which is the expected direction of correction for a small model applied to out-of-distribution web pages. The reliability of this calibration should itself be measured with a reliability diagram (predicted confidence vs. observed accuracy, bucketed) and reported as a expected calibration error (ECE) figure once real evaluation data exists – this document does not claim a specific ECE value, since none has been measured yet.

8.3 Change-Detection / Dirty-Region Metric

Given the previous Screen Graph $G_{(t-1)}$ and the newly observed graph G_t , a node n is marked changed if any of the following hold, each individually cheap to compute:

$$\begin{aligned} \Delta\text{bbox}(n) &= ||\text{bbox}_t(n) - \text{bbox}_{(t-1)}(n)||_2 > \epsilon_{\text{pos}} \\ \text{OR} \quad &\text{text}_t(n) \neq \text{text}_{(t-1)}(n) \\ \text{OR} \quad &n \in G_t \text{ AND } n \notin G_{(t-1)} \text{ (new node)} \\ \text{OR} \quad &n \in G_{(t-1)} \text{ AND } n \notin G_t \text{ (removed node)} \end{aligned}$$

where ϵ_{pos} is a small pixel-distance tolerance that absorbs sub-pixel rendering jitter without triggering unnecessary re-analysis. The set of changed nodes defines a minimal bounding region (or a small number of disjoint regions) which is the only area passed to the vision fallback stage, when vision is needed at all. We also apply a stability/debounce condition – a region is only considered stable and eligible for analysis once it has shown zero changes for a short window Δt_{stable} (in the low hundreds of milliseconds) – specifically to absorb visual noise such as loading spinners and auto-updating widgets, which would otherwise cause the event detector to fire continuously and defeat the purpose of event-driven processing.

8.4 Token and Compute Efficiency Model

Let F be the total number of observed frames in a session, and F_{vision} the number of frames on which the vision model was actually invoked (as opposed to being resolved entirely by AXTree/DOM, or skipped because no meaningful change was detected). We define the vision invocation rate as:

$$V_{\text{rate}} = F_{\text{vision}} / F$$

This single number is the honest, measurable version of the claim "lightweight." It should be reported from real session logs during development (for example, "the vision model was invoked on 4% of observed frames during a two-minute mixed-browsing session") rather than asserted qualitatively. Token efficiency for cloud calls follows the same logic: let S_{full} be the size in tokens of a naive full-page-context payload, and S_{sent} the size of the delta-only, minimised, structured payload actually transmitted. The compression ratio is:

$$\rho = 1 - (S_{\text{sent}} / S_{\text{full}})$$

reported as a percentage reduction. As with V_{rate} , this should be measured against real captured pages during development rather than assumed.

8.5 Privacy Risk Score

For the privacy validator gate, we define a simple, auditable risk score over the set of nodes N in a candidate outbound payload:

$$\text{Risk}(N) = \max_{n \in N} w(n) \cdot P_{\text{PII}}(n)$$

where $P_PII(n)$ is the PII detector's confidence that node n contains sensitive content, and $w(n)$ is a severity weight (for example, a detected financial account number is weighted higher than a detected first name). The payload is blocked outright if $Risk(N)$ exceeds a fixed threshold $\tau_privacy$, deliberately set conservatively (low), because the cost of a false block (one extra local retry) is far smaller than the cost of a false send (a genuine data leak). Using a max, rather than an average, over nodes is a deliberate choice: a payload should not be allowed to pass just because most of it is clean if even one node is a high-confidence, high-severity leak.

8.6 Two-Tier Escalation as a Constrained Optimisation

Section 10 commits to two local vision tiers of the same architecture family (Tier A, ~450M parameters; Tier B, ~1.6B parameters) plus an optional cloud tier. The routing function in Section 8.1 generalises directly to three tiers by chaining the same confidence test: Tier A runs first on every frame that reaches vision at all; if its calibrated confidence clears τ_high , its action is taken directly; otherwise Tier B runs on the same cropped region; if Tier B also fails to clear τ_high , the minimal redacted context escalates to the cloud tier. This can be written as one governing objective the whole staged design is optimising for:

$$\begin{aligned} \text{minimise} \quad E[\text{Cost}] &= c_A \cdot F + c_B \cdot F_B + c_cloud \cdot F_cloud \\ \text{subject to} \quad &\text{Accuracy} \geq \text{Accuracy_min} \end{aligned}$$

where F is the total number of frames reaching the vision stage, $F_B \leq F$ is the number escalated to Tier B, $F_cloud \leq F_B$ is the number further escalated to the cloud, and c_A , c_B , c_cloud are the respective per-invocation costs (latency and/or compute), with $c_A < c_B < c_cloud$ by construction. Because escalation only ever moves a frame to a strictly more expensive tier when the cheaper tier's calibrated confidence says it is warranted, this objective is minimised subject to an accuracy floor rather than traded off against it blindly - the system never spends Tier B or cloud cost on a frame Tier A could already answer confidently, and it never silently accepts a low-confidence Tier A answer just to save cost. The practical, measurable version of this objective is the escalation-rate pair $(F_B / F, F_cloud / F_B)$, which should be logged and reported alongside the vision invocation rate V_rate defined in Section 8.4.

9. Technology Stack

Every item in this stack is a real, currently available browser API or library, chosen specifically because it is small, well-supported, and directly matches a component of the architecture in Section 5. No item below is hypothetical or aspirational beyond normal engineering integration effort.

Layer	Technology	Why it was chosen
Extension shell	Chrome Extension Manifest V3 / Firefox WebExtensions	Standard cross-browser extension APIs; content-script access to DOM and page context
Structure source	Browser Accessibility Tree API + DOM API	Already computed by the browser for assistive technology; zero additional inference cost
GPU inference runtime	ONNX Runtime Web (onnxruntime-web), WebGPU execution provider	Production-maturity JavaScript runtime for ONNX models in-browser; WebGPU backend gives near-native GPU performance inside a browser tab
Vision Tier A (default, always-on path)	LFM2.5-VL-450M, ONNX-quantised (~770MB)	Smallest deployable member of the LFM2.5-VL family; same architecture lineage as Tier B, so behaviour is predictable and comparable across tiers; carries almost all frame-level vision load
Vision Tier B (rare escalation path)	LFM2.5-VL-1.6B, ONNX-quantised (~1.5GB)	Invoked only when Tier A's calibrated confidence falls below threshold; stronger reasoning on dense or ambiguous visual content, at a cost paid only occasionally
Independent benchmark comparator	SmoVLM-500M (Hugging Face)	A separate architecture lineage from the LFM2.5-VL family; run head-to-head against Tier A on our own UI-understanding evaluation set during build week, and the better-measured model is kept - this is a build-time decision backed by numbers, not a spec-sheet guess
Zero-download reference point (not a dependency)	Chrome built-in AI - Gemini Nano via the native Prompt API (multimodal image input)	Ships inside Chrome itself with no model download; cited as external validation that browser vendors are converging on the same on-device idea, not relied upon for the live demo since it is still flag-gated / early-preview and tuned for summarisation rather than dense visual grounding
Task-specific local models / embeddings	Transformers.js	JavaScript port of the Hugging Face transformers pipeline API; used for lightweight classification, embeddings, and any smaller auxiliary models beyond the main VLM
OCR (text-in-image PII catch)	Tesseract.js / PaddleOCR-web (WASM)	Runs on WebAssembly without requiring GPU; used only on small cropped regions, not full frames
Fallback compute layer	WebAssembly (WASM)	Used automatically when WebGPU is unavailable, per the degradation ladder in Section 15
Frontend / AI cursor UI	Standard HTML/CSS/JS injected via content script	Renders the visible cursor overlay, action highlights, and local audit-trail panel
Local storage	IndexedDB (browser-native)	Stores local preference memory, calibration parameters, and the local audit/replay log; never leaves the device
Cloud fallback model (optional, minimal-context only)	A hosted vision-language model reachable over HTTPS, invoked only on the token-optimised minimal payload	Receives only redacted, minimised, delta-only structured JSON, never a raw screenshot or full DOM

9.1 Browser Support Considerations

WebGPU is supported by default in current versions of Chrome, Edge, and Firefox on desktop platforms, and in Chrome on Android; Safari has also shipped WebGPU support by default in recent versions. Coverage is therefore broad but not universal across every device a user might have, which is precisely why the degradation ladder in Section 15 exists as a first-class part of the design rather than an afterthought.

10. Model Selection and Comparative Analysis

This section commits to a specific, two-tier model plan and justifies it against real alternatives, because "a lightweight local model" without a name and a routing strategy is one of the fastest ways a submission loses credibility with technical judges. All models discussed below are real, publicly documented models as of 2026; no fabricated benchmark scores are presented.

10.1 The Governing Decision: Tier, Don't Pick One Model

A single-model design forces an unwinnable choice: pick the small model and accept weaker reasoning on hard cases, or pick the large model and pay its full cost on every single frame, including the easy ones that did not need it. Our design refuses that trade-off by using two models from the same architecture family at two different sizes, connected by the calibrated confidence router already defined in Section 8.1:

Tier	Footprint	When it runs	Trigger
Tier A - LFM2.5-VL-450M	~770 MB	Default, always-on path	Every frame that reaches the vision stage at all (Section 5.2.3) starts here
Tier B - LFM2.5-VL-1.6B	~1.5 GB	Rare escalation path	Only when Tier A's calibrated confidence $C(a,t)$ falls below the low threshold (Section 8.1) - a small minority of frames
Cloud VLM	N/A (remote)	Rarest path, minimal payload only	Only when Tier B itself remains uncertain, and only with the redacted, token-optimised context of Section 5.2.9

Because Tier A and Tier B are the same LFM2.5-VL architecture family at different scales, their failure modes and confidence behaviour are far more predictable relative to each other than two unrelated architectures would be - a genuine engineering reason to prefer this pairing over picking two unrelated small models.

10.2 Comparison Against Alternatives

Model	Parameters	Browser-deployable today?	Footprint	Role in our design
LFM2.5-VL-450M (Tier A, selected)	~0.45B	Yes, ONNX + WebGPU build	~770MB	Best default-path balance: same family as the escalation tier, purpose-built for browser deployment
LFM2.5-VL-1.6B (Tier B, selected)	~1.6B	Yes, ONNX + WebGPU build	~1.5GB	Reserved for the minority of hard frames Tier A escalates; not used as the default path
SmolVLM-500M (benchmarked, not default)	~0.5B	Yes, community ONNX/WebGPU ports exist	Comparable to Tier A	Different architecture lineage; kept only as an independent build-time comparison point against Tier A on our own evaluation set

Model	Parameters	Browser-deployable today?	Footprint	Role in our design
Qwen2-VL / Qwen2.5-VL small variants	2B-3B+	Runnable via ONNX Runtime Web with conversion effort	Larger than either of our tiers	Strong general capability, but heavier and requires more manual export/quantisation work than a published browser build
Full cloud VLM (e.g. a large hosted multimodal model)	10s-100s of billions	No, server-only	N/A (remote)	Strongest raw capability, but requires sending data off-device on every call - exactly what this problem statement asks to minimise
Chrome built-in Gemini Nano (Prompt API, multimodal)	Undisclosed, browser-managed	Ships with Chrome, zero app-side download	Managed by Chrome, not our bundle	Strong external validation that browser vendors are converging on this exact idea; not used as a dependency because it is still flag-gated/early-preview and tuned for summarisation rather than dense visual grounding - cited in the pitch, not relied on in the build

10.3 Why This Is a Practical, Buildable Ceiling, Not a Compromise

The tiered design is deliberately the most aggressive optimisation that is still honestly buildable in a two-week window. Going further - for example, replacing Tier A with a distilled sub-100M model - would cut footprint further but risks pushing too much traffic into the expensive Tier B/cloud path, defeating the purpose. Stopping at a single 1.6B model, conversely, wastes capacity on the easy majority of frames. Two tiers of the same family, joined by one already-specified routing formula, is the smallest architecture that still has a real answer for hard visual cases - which is exactly what "lightweight and powerful" has to mean in practice, rather than as a slogan.

10.4 Honesty About What Is Not Yet Verified

This document deliberately does not claim measured latency, accuracy, or resource-usage numbers for this specific integration, because none have been measured yet at the time of writing. What can be stated truthfully is that the underlying components - ONNX Runtime Web's WebGPU execution provider, and Liquid AI's officially published ONNX-quantised builds of the LFM2.5-VL family - are real, currently available technologies documented by their respective publishers as working in-browser. Section 15 sets out the concrete plan to produce real, measured numbers (latency per invocation, per-tier escalation rate, memory footprint, PII precision/recall) during the build window, and these measured numbers, not projections, are what should be presented at the finale.

11. Privacy and Security Design

11.1 PII Detection Pipeline

Detection runs in layered stages, from cheapest and most reliable to most expensive and most general, so that the majority of sensitive content is caught without ever needing the vision model at all:

Stage	Detects	Cost / notes
1. DOM attribute check	type="password", type="email", type="tel", autocomplete attributes, explicit data-sensitive flags	Near-zero cost; deterministic; catches the majority of structured form-based PII
2. Pattern / regex + lightweight NER	Email addresses, phone numbers, government ID formats, card-number patterns, address-like text in resolved text nodes	Cheap; well-understood false positive/negative trade-offs that can be tuned
3. OCR-based text catch	Text embedded inside images or canvas-rendered regions, run through the same regex/NER pass after extraction	Only run on cropped regions already flagged as unresolved by structure sources
4. Vision-based detection	Faces, ID/document-shaped regions, handwritten content	Most expensive stage; only reached for regions no cheaper stage could clear
5. Structural-leak filter	Schema-level patterns (e.g. an internal admin tool's exact field layout) that are sensitive even with zero literal PII present	Operates on the Screen Graph shape itself, not on text content

Every stage writes its findings onto the corresponding Screen Graph node's *sensitive* field (Section 6.1) rather than modifying the payload directly, so redaction and validation both operate on the same structured evidence trail, which is also what powers the local audit panel described below.

11.2 Indirect Prompt Injection Defense

An agent that reads a web page's content and feeds it into a reasoning model is, by construction, accepting untrusted input into that model's context. A malicious or compromised page can include text that is invisible to a human user (zero-opacity elements, off-screen positioning, microscopic font size) but fully readable by the agent's extraction pipeline, phrased as an instruction rather than as content – for example, text engineered to make the agent click a hidden "subscribe" button or submit a saved payment method. This is a documented attack class against LLM-based browser agents generally, distinct from the user-data privacy concerns discussed elsewhere in this document: here, the threat is the environment attacking the agent, not the agent leaking the user's data.

Our defense has three parts. First, extracted page text is always passed into the reasoning context labelled and structurally separated as *data*, never concatenated with the user's own instruction in a way that could be confused with it. Second, an injection filter scans extracted text for imperative, system-prompt-like phrasing patterns (direct commands, role-reassignment attempts, instructions to ignore prior context) before that text is allowed into the reasoning step at all; flagged content is stripped from the payload. Third, any action whose target or intent was influenced by a flagged region is automatically routed to the confirmation band described in Section 8.1, rather than being allowed to auto-execute, regardless of how high the model's raw confidence score was. This is a genuinely uncommon safeguard in comparable browser-agent projects, and is one of the differentiators discussed in Section 13.

11.3 Local Audit and Replay Trail

Because the entire pipeline runs on-device, every stage's output can be logged locally without any privacy cost. The system maintains a local, inspectable record for each action the agent takes: what was captured, which nodes were flagged sensitive and redacted, what (if anything) was sent to the cloud, and why the confidence router made its decision. This turns the privacy claims made throughout this document from an assertion into something a user, or a judge, can literally open and inspect after any session.

12. Mind Map: Concept Overview

The diagram below consolidates the full design into six thematic clusters, showing how the individual mechanisms described in Sections 5 through 11 relate to the six governing principles of the system.

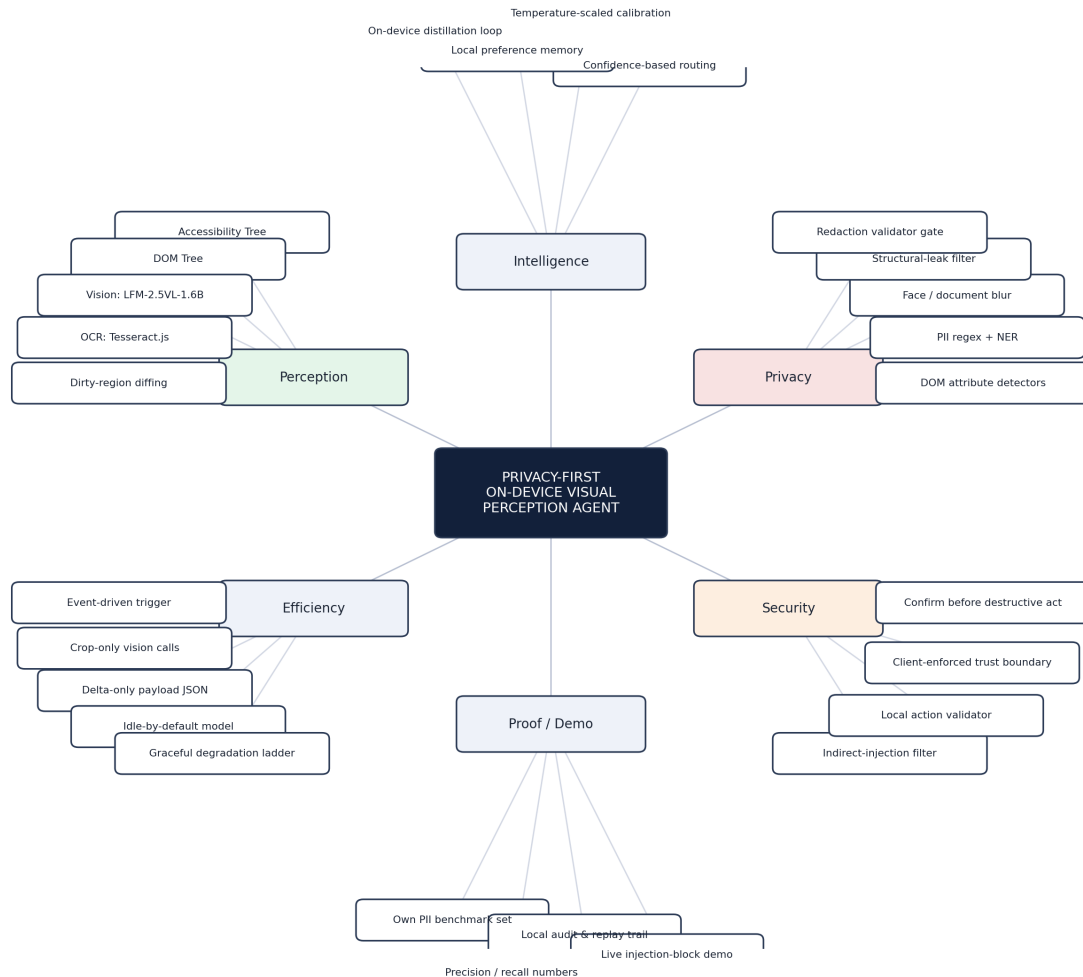


Figure 3. Concept mind map of the On-Device Visual Perception Agent.

13. Uniqueness and Differentiation From Other Submissions

The base architecture of "local perception, redact PII, send minimal structure to the cloud" is effectively suggested by the problem statement's own description, and a large fraction of competing teams will independently converge on some version of it. Genuine differentiation has to come from the specific, uncommon technical choices layered on top of that expected shape. The table below states plainly which parts of this design are the expected answer and which parts are the differentiators.

Design element	Category	Why
Local perception before cloud call	Expected	Directly implied by the problem statement's own wording
PII redaction before sending	Expected	Directly implied by the problem statement's own wording
Accessibility Tree as a primary, free structure source	Differentiator	Rarely used in comparable agent designs despite being computed by the browser at zero cost
Dirty-region diffing (only re-analyse changed regions)	Differentiator	Borrowed from graphics-engine rendering, not a common pattern in agent literature
Crop-only vision invocation, measured invocation rate	Differentiator	Turns "lightweight" from an adjective into a provable, measured number
Two-tier same-family vision escalation (450M default, 1.6B rare escalation)	Differentiator	Avoids both failure modes of a single-model design: never pays large-model cost on easy frames, never silently accepts a low-confidence answer to save cost
Explicit calibration method (temperature scaling) for routing confidence	Differentiator	Most teams will assume raw model confidence is trustworthy without addressing calibration at all
Hard privacy validator gate that can block a send outright	Differentiator	Most designs treat redaction as best-effort rather than as an enforced gate
Indirect prompt injection defense	Differentiator	Almost never addressed in student-level browser-agent projects; a real, documented threat class
Local, inspectable audit/replay trail	Differentiator	Converts privacy claims into something provably demonstrable rather than asserted
Self-built PII precision/recall benchmark	Differentiator	Most teams will demo qualitatively; real numbers are rare and highly credible to judges
Graceful degradation ladder for devices without WebGPU	Differentiator	Addresses hardware heterogeneity most demos ignore until directly asked

The recommended framing for a live pitch follows directly from this table: open with the differentiators (injection defense demo, measured vision invocation rate, the audit trail), and only describe the expected local-perception-plus-redaction architecture during technical question-and-answer, where it serves as solid foundation rather than the headline.

14. Alignment With SIH Evaluation Rubric

The problem statement's own evaluation guidance weights five criteria. The table below maps each one to the specific component of this design that addresses it, so that the connection between architecture and scoring is explicit rather than left for a judge to infer.

Rubric criterion	Weight	Addressed by
Visual context accuracy	25%	Screen Graph fusion of AXTree + DOM + vision; LFM-2.5VL-1.6B as the vision fallback; explicit DOM-blind mode for canvas/shadow-DOM applications
PII precision / recall	20%	Five-stage layered PII detection pipeline (Section 11.1); self-built labelled benchmark set for measured, reported numbers
Redaction precision	20%	Node-level redaction on the typed Screen Graph with bounding-box accuracy, rather than blind text/pixel masking
Client resource usage	20%	Idle-by-default design; event-driven Live Mode; crop-only, diff-gated vision invocation; measured vision invocation rate as a reported metric
End-to-end latency	15%	Context minimisation, delta-only token-optimised payloads, and local-first routing all reduce round-trips; this is the lowest-weighted metric and should not receive disproportionate build time relative to visual accuracy and PII metrics

The two highest-weighted criteria, visual context accuracy and PII precision/recall, together account for 45% of the score. Build effort during the hackathon window should be prioritised accordingly: getting the vision fallback genuinely working on a hard, DOM-blind page, and producing a real, measured PII precision/recall number, matter more to the final score than further polish on the AI cursor or additional UI refinement.

15. Feasibility, Build Plan and Risk Assessment

15.1 Graceful Degradation Ladder

Not every device the agent might run on supports WebGPU, and not every environment can hold a 1.5GB model in memory. Rather than assuming ideal hardware, the system defines an explicit fallback ladder so it degrades predictably instead of failing outright:

Tier	Configuration	When used
1	Vision + DOM + AXTree, WebGPU	Full capability, target tier for a modern desktop browser
2	AXTree + DOM only, no GPU vision	Structure-only understanding; vision-dependent tasks degrade gracefully with a clear "limited perception" indicator to the user
3	OCR (WASM) + DOM	Used when even lightweight GPU inference is unavailable but WASM is
4	DOM only, maximal redaction	Worst-case client hardware; still functional for simple, well-structured pages
5	Full cloud fallback, maximal redaction	Reserved for the weakest devices; all context still passes through the privacy validator before leaving the device

15.2 Two-Week Build Plan

A realistic two-week build should not attempt to fully complete all seven subsystems in this document simultaneously. The plan below sequences work so that a genuinely working, demoable core exists early, with the harder and more time-uncertain components (vision model integration, calibration) given the largest time buffer.

Window	Focus	Rationale
Days 1-3	Extension shell; AXTree + DOM extraction; Screen Graph construction; diff engine; basic redaction regex layer; local audit-trail UI skeleton	These are well-documented browser APIs and standard data-structure work; achievable quickly, including with AI-assisted code generation
Days 4-7	Integrate LFM-2.5VL-1.6B via ONNX Runtime Web + WebGPU; wire crop-only invocation into the unresolved-region path; verify on both simple and canvas/shadow-DOM heavy pages	Highest technical risk in the plan; execution-provider configuration, quantisation, and memory behaviour issues are realistic and should be budgeted extra time
Days 8-10	Build the self-labelled PII benchmark set; implement the privacy validator gate; run and record actual precision/recall numbers; implement temperature-scaling calibration against a held-out validation set	Mostly scripting once the pipeline exists; this is the step most teams skip and the step that most strengthens credibility with judges
Days 11-12	Implement the indirect-injection filter; construct one or two deliberately crafted adversarial test pages; rehearse a live block demonstration	Does not need to be exhaustive or bulletproof; needs to work convincingly on the prepared demonstration case
Days 13-14	Scope cut, integration testing, pitch rehearsal, buffer for the inevitable last-minute failure	Reserve time deliberately; do not schedule new feature work this late

15.3 Risk Register

Risk	Likelihood	Impact	Mitigation
WebGPU model integration takes longer than planned	High	High	Start this stream on Day 4, not later; fall back to running vision only on static screenshots first, then wire into live crop pipeline once stable
Calibration is not meaningfully better than raw confidence with limited data	Medium	Medium	Report the calibration curve honestly either way; an honest "we measured X and it needs more data" is stronger than an unverified claim
Injection filter produces false positives on legitimate pages	Medium	Low	Scope the filter to the one or two prepared demo pages for the live show; keep detection conservative elsewhere
Team runs out of time and must cut scope	High	Medium	Cut UI polish (AI cursor animation, extra dashboards) before cutting the PII benchmark numbers or the vision fallback – those two map to the highest-weighted rubric criteria
Judges test on a page/device outside what was validated	Medium	Medium	The degradation ladder (15.1) exists specifically so an unsupported case fails predictably and explainably rather than silently

16. Glossary of Abbreviations

Term	Meaning
AXTree	Accessibility Tree - browser-computed structural representation of a page for assistive technology
DOM	Document Object Model - the browser's live tree representation of page HTML
VLM	Vision-Language Model - a model that jointly processes images and text
ODVPA	On-Device Visual Perception Agent - the system proposed in this document
ONNX	Open Neural Network Exchange - an open format for representing trained ML models
WebGPU	A browser API giving JavaScript direct, compute-oriented access to the device GPU
WASM	WebAssembly - a portable, near-native-speed compute format that runs in browsers
OCR	Optical Character Recognition - extracting text from images or rendered pixels
NER	Named Entity Recognition - identifying structured entities (names, numbers, etc.) in text
PII	Personally Identifiable Information
ECE	Expected Calibration Error - a standard metric for how well a model's confidence matches its true accuracy
IoU	Intersection over Union - a standard metric for bounding-box overlap accuracy
SPA	Single-Page Application - a web app that renders and updates without full page reloads
SIH	Smart India Hackathon
ISRO	Indian Space Research Organisation

17. References and Sources

The following sources were used to verify that the specific technologies named in this document are real and current as of 2026. This document does not reproduce text from these sources verbatim; all technical descriptions above are written independently.

- ONNX Runtime Web - official package documentation, npm registry (onnxruntime-web).
- Microsoft Open Source Blog - "ONNX Runtime Web unleashes generative AI in the browser using WebGPU."
- ONNX Runtime documentation - "Using WebGPU" execution provider guide, onnxruntime.ai.
- Liquid AI - LFM-2.5VL-1.6B model documentation and Hugging Face model card (LiquidAI/LFM2.5-VL-1.6B).
- Hugging Face - Transformers.js v3 blog post on WebGPU-accelerated in-browser model execution.
- Smart India Hackathon 2026 - Problem Statement 26171 official listing, Indian Space Research Organisation, Department of Space.

All model names, parameter counts, and deployment details referenced in Sections 9 and 10 reflect publicly published information available at the time this document was written. Teams should re-verify current model versions and browser support tables close to the hackathon date, since this is a fast-moving area of the ecosystem.

18. Conclusion

Problem Statement 26171 asks for an on-device visual perception layer that lets a browser agent understand a user's screen without exporting sensitive data to a server, within the resource constraints of a real client device. The architecture in this document answers that requirement directly: a typed Screen Graph built primarily from data the browser already computes for free (the Accessibility Tree and DOM), a real, currently deployable on-device vision-language model (LFM-2.5VL-1.6B via ONNX Runtime Web on WebGPU) invoked only on the cropped, changed regions that structure alone cannot explain, a layered privacy pipeline with a hard validator gate, a calibrated confidence router that decides when local perception is sufficient, and a defense against indirect prompt injection that addresses a real security dimension of agentic web browsing largely absent from comparable projects.

The base shape of this architecture – local-first perception with cloud fallback – is close to what the problem statement itself implies, and should be expected to appear, in some form, across many competing submissions. What is intended to distinguish this submission is not the shape of the pipeline but the specific, verifiable engineering choices inside it: a free structural source most teams will not think to use, a provable measured invocation rate instead of an unverified efficiency claim, a security threat model most teams will not address at all, and a commitment to reporting real precision/recall numbers on a self-built benchmark rather than qualitative assurance.

Whether this idea succeeds at the finale depends, in the end, on execution: how much of this design is genuinely working and measured by demo day, and how clearly that work is communicated to judges in the limited time of a pitch. This document, and the two-week build plan in Section 15, are written to make that execution as achievable and as honestly scoped as possible.

End of document.